Microsoft Purview Administrator Training

Implementation Solution Guide

Version 1.0.0 · 2026-07-07

Contents

Executive Summary — Microsoft Purview Administrator Training 4

What you get . . . 4

What this delivers . . . 4

A note on scope: skills training, not a governance determination . . . 5

1. Solution Overview — Microsoft Purview Administrator Training 5

The problem this solves . . . 5

Who this is for . . . 5

What the curriculum covers . . . 6

What "training" delivers here (the model, applied twice) . . . 6

The training naming convention (used verbatim throughout) . . . 7

Benefits . . . 7

Limitations (be honest with buyers) . . . 8

The deployment model . . . 8

2. Requirements Guide — Microsoft Purview Administrator Training 8

2.1 A training environment — sandbox tenant preferred, scoped production acceptable . . . 8

2.2 A training scope group and naming convention . . . 9

2.3 Least-privilege Purview role groups for the trainees . . . 9

2.4 Security & Compliance PowerShell + Microsoft Graph toolchain . . . 9

2.5 Licensing sufficient for the modules in scope (verify per feature) . . . 9

2.6 Illustrative lab data and scope (not real regulated data) . . . 10

2.7 Stakeholders identified for the hand-off (compliance, records, privacy, HR, legal) . . . 10

2.8 A pilot scope and change control for the training rollout . . . 10

Global Requirements — Microsoft Purview Solutions 10

1. Microsoft 365 tenant with the required Purview/compliance licensing . . . 11

2. Purview roles (least-privilege) and portal access . . . 11

3. Security & Compliance PowerShell + Graph toolchain . . . 11

4. Data sources & scope in scope . . . 12

5. Compliance, records, privacy & legal stakeholders . . . 12

6. A pilot scope and change control . . . 12

3. Implementation Guide — Microsoft Purview Administrator Training 13

Phase A — Stand up the training program . . . 13

Phase B — Foundations . . . 14

Phase C — Information protection labs . . . 14

Phase D — Data Loss Prevention lab . . . 15

Phase E — Data lifecycle & records lab . . . 16

Phase F — HR/privacy-sensitive labs (policy-model walkthroughs) . . . 16
Phase G — Audit, eDiscovery, DSPM . . . 17
Phase H — Validate competence and operationalize . . . 18
Rollback summary . . . 18
5. Testing Guide — Microsoft Purview Administrator Training 19
Test 1 — Readiness: connectivity, scope, and feature visibility . . . 19
Test 2 — Scope safety: training group contains only test accounts . . . 19
Test 3 — Lab 1 Labels: label published to the training scope only . . . 19
Test 4 — Lab 4 Auto-labeling: exists and is in SIMULATION (never enabled) . . . 20
Test 5 — Lab 2 DLP: exists in TEST mode and enforced nothing . . . 20
Test 6 — Lab 3 Retention: exists, scoped, illustrative period . . . 20
Test 7 — Modules 5–6: privacy controls and role separation (no real data) . . . 20
Test 8 — Module 7 Audit: search returns and exports . . . 20
Test 9 — Module 8 eDiscovery: training custodian on hold, search returns . . . 20
Test 10 — Competency roll-up: every attempted lab PASSes or is documented . . . 21
Test 11 — Teardown: tenant returns to the Step 3 baseline . . . 21
6. Troubleshooting Guide — Microsoft Purview Administrator Training 22
A. Connectivity & toolchain . . . 22
B. Scope group problems (`TRAIN-Pilot-Users`) . . . 22
C. Lab 1 Labels . . . 23
D. Lab 2 DLP — the critical "never enforce" checks . . . 23
E. Lab 4 Auto-labeling — simulation only . . . 23
F. Lab 3 Retention . . . 24
G. Modules 5–6 Insider Risk / Communication Compliance . . . 24
H. Module 7 Audit . . . 25
I. Module 8 eDiscovery . . . 25
J. Validation / competency roll-up . . . 25
When to escalate to the governance functions (not a "fix") . . . 25
7. Best Practices — Microsoft Purview Administrator Training 26
8. Deliverables Summary — Microsoft Purview Administrator Training 27
Documentation . . . 27
PowerShell scripts (Security & Compliance PowerShell / Microsoft Graph — safe by default) . . . 27
What the buyer receives . . . 28
Scope note . . . 29
Outcome . . . 29
Appendix A — PowerShell Scripts 30

Executive Summary — Microsoft Purview Administrator Training

The people who operate your **Microsoft Purview** controls — sensitivity labels, data loss prevention, data lifecycle and records, insider risk, communication compliance, audit, eDiscovery, and data security posture management — often learned the platform by **trial and error in the live tenant**. Purview is broad, heavily **licensing-dependent**, and **actively evolving**, so ad-hoc learning is slow, inconsistent between administrators, and genuinely risky on the high-impact features. The result is a **skills gap that is also a compliance gap**.

This is a complete, deploy-it-in-order **training curriculum** that closes that gap deliberately. It is a sequence of **hands-on labs** — one per Purview solution area — that each build a capability in a **safe sandbox scope and in simulation/test mode first**, end at an explicit **pass/fail validation gate**, and have a **teardown** that returns the tenant to a known baseline. It is written for a competent M365 administrator (or a team) who needs to **own day-to-day Purview operations**, and it is self-guided, tested, and reusable for every new hire.

What you get

• A plain-language **Solution Overview** of the training model — a lab-and-runbook curriculum that teaches every Purview area **scope-first, simulate-first, validate-before-enforce**, so the safe- rollout discipline transfers straight to production.

• A **Requirements Guide** with verify / implement-if-missing for every prerequisite — a training environment (sandbox preferred), a `TRAIN-Pilot-Users` scope group of test accounts, least-privilege role groups, the PowerShell/Graph toolchain, per-module licensing, illustrative lab data, and the compliance/records/privacy/HR/legal stakeholders for the hand-off.

• An **Implementation Guide** of precise, ordered steps: stand up the program (readiness → scope → baseline), run each lab in the training scope and simulation/test mode with a named object, a pass/fail gate, and a teardown, then convert the labs into **operational runbooks**.

• **Five PowerShell scripts** — a readiness/health helper, a scope-group configuration helper (default `-WhatIf`), a baseline discovery snapshot, a per-lab validation checker, and a competency report — read-only by default, no secrets.

• **Testing and Troubleshooting** guides with explicit validation gates — the most important being that a **training DLP or auto-labeling policy is never left enforcing** — plus **Best Practices** for building durable, evidenced competence.

What this delivers

A Purview administration team that has **built and proven** competence across the whole platform — every capability practiced safely on **illustrative** data in a scoped, simulated sandbox, validated at a pass/fail gate, and turned into runbooks — so operating Purview is a deliberate, evidenced skill instead of learning on a live fire, and onboarding the next administrator is a repeatable process.

A note on scope: skills training, not a governance determination

This package builds the *technical* operator. Whether a retention schedule, records file plan, legal hold, or an insider-risk / communication-compliance **case outcome** is *correct* is a determination for your **compliance, records, privacy, HR, and legal** functions — engaged here as named stakeholders. The labs teach the method and produce the evidence; the governance decision stays with the governance owners.

1. Solution Overview — Microsoft Purview Administrator Training

The problem this solves

Most teams learn to run **Microsoft Purview** by trial and error in a live tenant — clicking through the portal on a real incident, guessing at label scope, publishing a DLP policy straight to enforcement, and discovering the blast radius afterward. Purview is broad (sensitivity labels, DLP, data lifecycle & records, insider risk, communication compliance, audit, eDiscovery, and data security posture management), heavily **licensing-dependent**, and **actively evolving** — so trial-and-error is slow, inconsistent between admins, and, on high-impact features, genuinely risky. The result is a **skills gap that is also a compliance gap**: the people who operate your governance controls have no structured way to build and prove competence.

This package fixes that with a **structured Purview administrator training curriculum** — a sequence of **hands-on labs** that each build a Purview capability in a **safe sandbox scope and in simulation/test mode first**, plus **operational runbooks** for the recurring work an administrator does after go-live. Every lab is named object-for-object, gated with an explicit **pass/fail validation**, and has a **teardown** so the training environment returns to a known state. It is self-guided, tested, and repeatable — competence built deliberately instead of learned on a live fire.

> ***Vendor boundary.*** *The **Microsoft Purview portal** (`purview.microsoft.com`), the **Microsoft 365 admin center**, **Security & Compliance PowerShell** (`Connect-IPPSSession`), and **Microsoft Graph** are authored at current, exact detail from Microsoft's public documentation. Purview capabilities are heavily **licensing-/plan-dependent** (many require **Microsoft 365 E5** or the **E5 Compliance** add-on; some are in preview) and the portal is **evolving** — availability and portal paths are **cited or flagged "verify your licensing/tenant,"** never invented. This is **administrator skills training**, not a records, privacy, or legal determination: retention schedules, records file plans, legal holds, and insider-risk / communication-compliance **case outcomes** are **deferred to the organization's compliance, records, privacy, and legal functions** — the labs teach the *technical method* and produce evidence, not the governance decision.*

Who this is for

A competent Microsoft 365 administrator (or a team of them) who needs to **own day-to-day Purview operations** — new to Purview, cross-training from Exchange/security administration, or formalizing skills they picked up ad hoc. The curriculum assumes M365 admin fundamentals and builds Purview specifics from the ground up.

What the curriculum covers

The labs are organized into modules that mirror the Purview solution areas. Each is a **self-contained lab** with an objective, prerequisites, ordered steps (in a sandbox scope, simulation/test first), a validation gate, and a teardown.

Module	Lab focus	Runs in
0 — Foundations	Portal orientation, **roles & least-privilege scopes**, PowerShell/Graph toolchain	Read-only
1 — Sensitivity labels	Create a training label + label policy scoped to a training group	Scoped pilot
2 — Data Loss Prevention	Author a DLP policy and run it in **test / simulation** mode (no enforcement)	Simulation
3 — Data lifecycle & records	Configure a retention policy/label against an *illustrative* schedule	Scoped pilot
4 — Auto-labeling	Stand up an auto-labeling policy in **simulation** mode and read the results	Simulation
5 — Insider Risk Management	Walk the policy model with **privacy-by-design** and role separation	Portal, limited scope
6 — Communication Compliance	Walk the policy model with **pseudonymization** and role separation	Portal, limited scope
7 — Audit	Run **Audit (Standard/Premium)** searches and export evidence	Read-only
8 — eDiscovery	Create a case, place/verify a hold on a training custodian, search	Scoped pilot
9 — DSPM & analytics	Read **Data Security Posture Management** and classification analytics	Read-only
10 — Operations	Turn the labs into **runbooks**: alert triage, coverage review, change control	Ongoing

Modules 5 and 6 are HR/privacy-sensitive. They are taught as *policy-model walkthroughs* with **privacy-by-design, pseudonymization where the feature supports it, and strict role separation** — never with real employee data, and with all case *outcomes* deferred to HR/legal.

What "training" delivers here (the model, applied twice)

The house deployment model — **Assess** → **Design** → **Configure** → **Pilot (simulate/test)** → **Validate** → **Expand (enforce)** — is used two ways in this package:

1. **To roll out the training program.** *Assess* the team's current Purview skills and the tenant baseline; *design* the curriculum and the sandbox scope; *configure* a training scope group and naming convention; *pilot* the labs; *validate* competence with the pass/fail gates; *expand* into the operational runbooks and a recurring cadence.

1. **Inside every lab.** Each capability is built the same disciplined way an administrator must build it in production — **scoped to a training group and run in simulation/test/disabled mode first**, validated against a pass/fail gate, and only then understood as "ready to enforce." The trainee *practices the safe rollout pattern itself*, so the habit transfers to production.

The training naming convention (used verbatim throughout)

Every object a lab creates is prefixed `TRAIN-` and scoped to the training group `TRAIN-Pilot-Users`, so training artifacts are unmistakable, auditable, and safe to tear down:

Lab	Object created	Named
1	Sensitivity label + policy	`TRAIN-Label-Confidential` in `TRAIN-LabelPolicy-Pilot`
2	DLP policy (test mode)	`TRAIN-DLP-PII-Sim`
3	Retention policy/label	`TRAIN-Retention-Pilot` / `TRAIN-RetLabel-Illustrative`
4	Auto-labeling policy (simulation)	`TRAIN-AutoLabel-Sim`
7	Audit search	`TRAIN-Audit-Search`
8	eDiscovery case	`TRAIN-eDiscovery-Case`

Benefits

• **Competence you can prove.** Every lab ends at a pass/fail validation gate, so "trained" means an evidenced capability, not a claim.

• **Safe by construction.** Labs run in a scoped training group and in simulation/test/disabled mode — trainees learn the high-impact features (DLP, labels, retention, auto-labeling) without risking production.

• **The safe-rollout habit transfers.** Practicing scope-first, simulate-first, validate-before-enforce in every lab builds the exact discipline production Purview work demands.

• **Runbooks, not just knowledge.** Module 10 converts the labs into repeatable operational runbooks (alert triage, coverage review, audit request handling) the team uses after training.

• **A fraction of consulting cost.** Self-guided, tested curriculum with PowerShell helpers — reusable for every new administrator you onboard.

Limitations (be honest with buyers)

• **This is skills training, not a governance determination.** The labs use *illustrative* label taxonomies, retention periods, and policy scopes so trainees can practice the mechanics. **Real** retention schedules, records file plans, legal holds, and insider-risk/communication-compliance outcomes are **legal/records/privacy/HR determinations** — deferred to those functions.

• **Licensing gates what you can practice.** Advanced DLP, auto-labeling, insider risk, communication compliance, Audit Premium, eDiscovery Premium, and DSPM require **E5 / E5 Compliance** (some preview). Where a feature is not licensed, the lab degrades to a read-only walkthrough — **verify your licensing/tenant.**

• **The portal evolves.** Purview paths and cmdlet behavior change; steps reflect current public documentation and are flagged **"verify in your tenant"** where volatile. No portal path or cmdlet is invented.

• **A sandbox/training tenant is strongly preferred.** Where labs run in a production tenant, they are strictly scoped to `TRAIN-Pilot-Users` and simulation/test mode, and torn down afterward.

The deployment model

> ***Assess skills & baseline → Design curriculum & sandbox scope → Configure the training scope → Pilot the labs in simulation/test → Validate competence at each pass/fail gate → Expand into operational runbooks.*** *The pattern is* ***Assess → Design → Configure → Pilot → Validate → Expand****, with an explicit gate per lab and a documented teardown for every training object created.*

2. Requirements Guide — Microsoft Purview Administrator Training

Verify **every** requirement before running the labs. Each entry follows **Description · Why · Verify · Implement-if-missing.** Category-wide prerequisites are published once in `global-requirements-purview.md` — this guide covers the prerequisites specific to standing up and running the **training curriculum** and does not repeat the six category requirements.

> ***Vendor boundary.*** *Microsoft Purview portal, M365 admin center, Security & Compliance PowerShell, and Microsoft Graph paths are authored from Microsoft's current public documentation. Purview capability availability is* ***licensing-dependent*** *and the portal* ***evolves*** *— flagged* ***"verify your licensing/tenant."*** *This is administrator training:* ***illustrative*** *taxonomies, retention periods, and policy scopes are used so trainees can practice;* ***real*** *records/retention/legal and HR/privacy determinations are deferred to the relevant governance functions.*

2.1 A training environment — sandbox tenant preferred, scoped production acceptable

Description: An environment to run the hands-on labs. A **dedicated sandbox/training/dev tenant** is strongly preferred; where labs must run in a production tenant, they are strictly confined to the training

scope group and simulation/test mode and torn down afterward. **Why:** Labs create real Purview objects (labels, DLP policies, retention policies, an eDiscovery case). A sandbox isolates that from production; scoped-and-simulated is the fallback discipline when it cannot. **Verify:** Confirm which tenant the team will train in, and that you have admin sign-in to it. Run `OPSTRAIN-Readiness.ps1` — it reports the connected tenant and whether the training scope exists. **Implement:** Provision or nominate a training tenant; if using production, agree the confinement rules (scope group + simulation/test + teardown) with change control before Lab 1.

2.2 A training scope group and naming convention

Description: A **security group** `TRAIN-Pilot-Users` containing only training/test accounts, and the `TRAIN-` object-naming convention used verbatim across every lab and script. **Why:** Labels, DLP, retention, and auto-labeling are all published/targeted to this group so a lab never touches a real user; the `TRAIN-` prefix makes every training artifact unmistakable and safe to tear down. **Verify:** In **Entra ID** → **Groups** (or `Get-MgGroup`) confirm `TRAIN-Pilot-Users` exists and contains only test accounts. `OPSTRAIN-Readiness.ps1` flags whether the group is present. **Implement:** Create the group with test accounts (`OPSTRAIN-LabScope.ps1 -Commit` can create/verify it), and reserve the `TRAIN-` naming convention before configuring anything.

2.3 Least-privilege Purview role groups for the trainees

Description: Each trainee holds the **least-privilege Purview role group** the module needs — e.g. **Information Protection** (labels), **Compliance Data Administrator / Compliance Administrator** (DLP, retention), **Records Management**, **Insider Risk Management** roles, **Communication Compliance** roles, **Audit** roles, **eDiscovery Manager** — with Global Administrator reserved and avoided. **Why:** Purview uses fine-grained role groups; a lab only works with the matching role, and training on least-privilege is itself a core competency. Insider Risk and Communication Compliance require **strict role separation** (analyst vs investigator vs admin) as a privacy control. **Verify: Purview portal** → **Settings** → **Roles & scopes / Permissions** — confirm each trainee's role group membership. `OPSTRAIN-Readiness.ps1` reports the connected admin's readable role context. **Implement:** Assign the least-privilege role group per module under change control; for Modules 5–6 assign the **separated** roles to different people. Do not grant Global Administrator "to make the lab work."

2.4 Security & Compliance PowerShell + Microsoft Graph toolchain

Description: A workstation with **ExchangeOnlineManagement** (for `Connect-IPPSSession`) and **Microsoft.Graph** PowerShell modules, PowerShell 5.1+ or 7+, and a modern-auth/MFA-capable admin account. **Why:** The labs and all five provided scripts run through Security & Compliance PowerShell and Graph; learning the cmdlet surface is part of the curriculum. **Verify:** `Connect-IPPSSession` and `Connect-MgGraph` both succeed; `Get-Label` returns without error. `OPSTRAIN-Readiness.ps1` performs these connectivity checks. **Implement:** `Install-Module ExchangeOnlineManagement` and `Install-Module Microsoft.Graph`; confirm modern-auth sign-in before Lab 0.

2.5 Licensing sufficient for the modules in scope (verify per feature)

Description: The **compliance licensing** each in-scope module requires — E3 for baseline labels and manual retention; **E5 / E5 Compliance** for advanced DLP, auto-labeling, insider risk, communication compliance, **Audit Premium**, **eDiscovery Premium**, and **DSPM** (some capabilities preview). **Why:**

Purview features are plan-gated; an unlicensed feature is not visible and its lab cannot run in full. Knowing *which* license unlocks *what* is itself an administrator competency. **Verify:** In the **M365 admin center** confirm subscriptions/add-ons; confirm each module's feature is visible in the Purview portal. `OPSTRAIN-Readiness.ps1` probes which feature areas are readable and warns where one is not (likely unlicensed). **Implement:** Assign the required licensing before the corresponding module, **or** run that module as a read-only walkthrough. **Verify your licensing/tenant** — do not assume availability.

2.6 Illustrative lab data and scope (not real regulated data)

Description: Illustrative sample content and an **illustrative** classification/label/retention scheme for the labs (e.g. a `TRAIN-Label-Confidential` label, a synthetic-PII test document for DLP) — explicitly **not** real regulated records, real employee communications, or a live records schedule. **Why:** Trainees must practice the mechanics on safe data; using real regulated data or a real schedule would turn a *training exercise* into a *governance action*, which is a determination reserved to the governance functions. **Verify:** Confirm the sample content is synthetic/non-sensitive and the taxonomy/periods are labeled "illustrative." Insider Risk / Communication Compliance labs use **no** real employee data. **Implement:** Prepare synthetic sample files and an illustrative taxonomy before the relevant labs; keep real schedules and real cases out of the training environment.

2.7 Stakeholders identified for the hand-off (compliance, records, privacy, HR, legal)

Description: The named **compliance, records-management, privacy, HR, and legal** stakeholders who own the real determinations the trained administrators will *implement* — retention schedules, records file plans, legal holds, insider-risk and communication-compliance policy and case outcomes. **Why:** Training builds the *technical* operator; the *governance* decisions stay with these functions. Modules 5–6 in particular are HR/privacy-sensitive and their outcomes are never the administrator's call. **Verify:** The owners are named and the hand-off (who decides vs who configures) is written down. **Implement:** Identify the owners before Module 10 so the runbooks route each real decision to the right function; capture the RACI in the operations runbook.

2.8 A pilot scope and change control for the training rollout

Description: A **pilot cohort** (the first trainees), **simulation/test/disabled mode confirmed available** for the high-impact labs, and a **change-control** path for any training object created in a shared tenant — mirroring the category-wide pilot/change-control requirement, applied to the *training program itself*. **Why:** The curriculum is delivered the same disciplined way it teaches: piloted, validated, then expanded — and any object created in a shared tenant follows change control and is torn down. **Verify:** A named first cohort, confirmed simulation/test availability for Modules 2 and 4, and an agreed change/teardown window. **Implement:** Define the cohort and change-control path before Lab 1; use each lab's teardown to leave the tenant clean.

Global Requirements — Microsoft Purview Solutions

These requirements apply to **every** solution in the **Purview** category (implementing and managing Microsoft Purview, sensitivity & retention labels, DLP, data lifecycle & records, insider risk, communication compliance, audit, data security posture management, and data governance). Verify each before starting a

solution's implementation steps.

> ***Vendor boundary (applies to every Purview solution).*** *The **Microsoft Purview portal** (purview.microsoft.com), the **Microsoft 365 admin center**, **Security & Compliance PowerShell** (`Connect-IPPSSession`), and **Microsoft Graph** are authored at exact, current click-by-click / cmdlet detail from Microsoft's public documentation. Microsoft Purview features are heavily **licensing-/plan-dependent** (many require **Microsoft 365 E5** or the **E5 Compliance** add-on; some are in preview) and the portal is **actively evolving** — capability availability and portal paths are **cited or flagged "verify your licensing/tenant"**, never asserted or invented. **Regulatory, records-retention, and legal determinations** (retention schedules, records file plans, legal holds, insider-risk case actions, regulatory applicability) are **deferred to the organization's compliance, records, privacy, and legal functions** — the solutions provide the technical configuration method and evidence, not the determination. Automation is **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets.*

1. Microsoft 365 tenant with the required Purview/compliance licensing

- **Description:** A Microsoft 365 tenant with the **compliance licensing** the capability requires

(E3 for baseline; **E5 / E5 Compliance** for advanced DLP, auto-labeling, insider risk, communication compliance, Audit Premium, DSPM, etc.).

- **Why required:** Most Purview capabilities are plan-gated; the feature won't be available without the

right plan.

- **How to verify:** In the M365 admin center confirm the subscription/add-ons; check the feature is

visible in the Purview portal.

- **How to implement if missing:** Assign/purchase the required licensing before starting.

2. Purview roles (least-privilege) and portal access

- **Description:** Access with the least-privilege **Purview / compliance role** the solution names

(e.g. Compliance Administrator, Compliance Data Administrator, Information Protection, Records Management, Insider Risk, eDiscovery, Audit roles), with Global Administrator only where required.

- **Why required:** Purview uses fine-grained role groups; configuration requires the matching role.
- **How to verify:** Purview portal → **Roles & scopes / Permissions** — confirm the role membership.
- **How to implement if missing:** Assign the least-privilege role group under change control.

3. Security & Compliance PowerShell + Graph toolchain

- **Description:** A workstation with **Security & Compliance PowerShell** (`Connect-IPPSSession` via the

ExchangeOnlineManagement module) and **Microsoft Graph PowerShell**, modern-auth / MFA-capable admin.

- **Why required:** Configuration, automation, and the provided scripts run through these.

- **How to verify:** `Connect-IPPSSession` and `Connect-MgGraph` succeed with the admin account.
- **How to implement if missing:** Install the modules and confirm modern-auth sign-in before running steps.

4. Data sources & scope in scope

- **Description:** The **workloads and data sources** in scope (Exchange, SharePoint, OneDrive, Teams, endpoints/devices, and — for data governance — the broader data estate) and a **data-classification / sensitivity** intent to drive labels, DLP, and retention.
- **Why required:** Purview policies key off the in-scope workloads and the classification scheme.
- **How to verify:** Confirm the in-scope workloads and the classification/sensitivity approach with stakeholders.
- **How to implement if missing:** Define the scope and classification intent before authoring policies.

5. Compliance, records, privacy & legal stakeholders

- **Description:** Engagement with the **compliance, records-management, privacy, security, and legal** stakeholders whose policies the configuration must satisfy (retention schedules, records file plans, regulatory obligations, insider-risk and communication-compliance policy, legal hold).
- **Why required:** Purview configuration must align to enterprise policy and legal/records requirements, not technical defaults; several capabilities have HR/legal sensitivity.
- **How to verify:** Confirm the applicable policies, the records schedule, and named approvers.
- **How to implement if missing:** Identify the owners and capture the requirements before configuring.

6. A pilot scope and change control

- **Description:** A **pilot scope** (users/sites/policies in simulation or a limited ring) to validate on before tenant-wide enforcement, and a **change-control** process — most Purview policies support a **simulation / test / disabled** mode that MUST be used before enforcement.
- **Why required:** Every solution follows an **Assess → Design → Configure → Pilot (simulate) → Validate → Expand (enforce)** model; DLP, labels, retention, and insider-risk changes are high-impact.
- **How to verify:** Confirm a named pilot scope, that simulation/test mode is available, and a change window/approval path.
- **How to implement if missing:** Define the pilot and agree change control before enforcing anything.

*The deployment model across every Purview solution is the same: **Assess** → **Design** → **Configure** → **Pilot (simulate/test)** → **Validate** → **Expand (enforce)**, with explicit pass/fail gates and a documented rollback. Automation is delivered as **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets. Regulatory/records/legal determinations are deferred to the relevant

governance functions.*

3. Implementation Guide — Microsoft Purview Administrator Training

Follow these steps **in order**, working in the **Microsoft Purview portal** (`purview.microsoft.com`), the **Microsoft 365 admin center**, and **Security & Compliance PowerShell** / **Microsoft Graph** from the training workstation.

> ***Model:*** *Stand up the training program first (readiness → scope → baseline), then run the labs in sequence.* ***Every lab is scoped to*** `TRAIN-Pilot-Users` ***and runs in simulation/test/disabled mode first****, ends at a* ***pass/fail validation gate****, and has a* ***teardown****. Name every object exactly as shown (*`TRAIN-…`*), verbatim across the portal, scripts, and reports.*
>
> ***Training, not determination.*** *The label taxonomy, retention periods, and policy scopes below are* ***illustrative*** *so trainees can practice the mechanics.* ***Real*** *retention schedules, records file plans, legal holds, and insider-risk / communication-compliance* ***case outcomes*** *are* ***legal/records/privacy/HR determinations*** *— configure to *illustrative* values here and route real decisions to those functions.*
>
> ***Verify in your tenant.*** *Purview capability availability is* ***licensing-dependent*** *and portal paths* ***evolve****. Where a feature is unlicensed, run that lab as a* ***read-only walkthrough****. Confirm exact toggles for your tenant; defer volatile specifics to Microsoft's documentation — nothing here is invented.*

Phase A — Stand up the training program

Step 1 — Readiness check (read-only)

1. From the training workstation, `Connect-IPPSSession` and `Connect-MgGraph` with the admin account.
2. Run `OPSTRAIN-Readiness.ps1`. It confirms connectivity, reports the connected **tenant**, the

readable **role context**, whether the training scope group `TRAIN-Pilot-Users` exists, and **which Purview feature areas are readable** (labels, DLP, retention, auto-labeling, audit) — warning where a feature is not readable (likely **unlicensed** → that lab becomes a read-only walkthrough).

1. **Gate:** connectivity succeeds and the readiness report lists the feature areas your curriculum will

cover. Resolve any missing module/role/license before continuing.

Step 2 — Create the training scope group (`TRAIN-Pilot-Users`)

1. Preview first: `OPSTRAIN-LabScope.ps1` (default `-WhatIf`) shows what it would create/verify.
2. Apply: `OPSTRAIN-LabScope.ps1 -Commit` creates (or verifies) the `TRAIN-Pilot-Users` security

group containing only **test accounts**, or confirm it in **Entra ID** → **Groups**.

1. **Gate:** `TRAIN-Pilot-Users` exists and contains only training/test accounts — **no real users.**

Step 3 — Capture the lab baseline (read-only)

1. Run `OPSTRAIN-LabBaseline.ps1` to snapshot the current Purview configuration — existing labels, label

policies, DLP policies/rules, retention policies, and auto-labeling policies — to a CSV baseline.

1. This is the **known-good "before"** state. Each lab's teardown returns the tenant to it, and

`OPSTRAIN-LabValidate.ps1` diffs against it to confirm only `TRAIN-` objects were added.

1. **Gate:** the baseline CSV is written and reviewed; the team knows what existed before training.

Phase B — Foundations

Step 4 — Module 0: Portal orientation, roles & toolchain (read-only)

1. Tour the **Purview portal** (`purview.microsoft.com`): the solution catalog (Information Protection,

Data Loss Prevention, Data Lifecycle Management, Records management, Insider Risk Management, Communication Compliance, Audit, eDiscovery, DSPM) and **Settings** → **Roles & scopes / Permissions**.

1. Have each trainee locate their assigned **least-privilege role group** and explain what it grants —

and, for Modules 5–6, why analyst/investigator/admin roles must be **separated**.

1. Confirm the toolchain: `Connect-IPPSSession`, `Connect-MgGraph`, and `Get-Label` all succeed.

2. **Gate:** each trainee can navigate the portal, name their role and its least privilege, and connect

both PowerShell endpoints. **Rollback:** none (read-only).

Phase C — Information protection labs

Step 5 — Lab 1: Sensitivity label + label policy (scoped pilot)

1. **Purview portal** → **Information Protection** → **Labels** → **Create a label**: create

`TRAIN-Label-Confidential` (illustrative — display name, tooltip; optional encryption/marking left as a discussion point, not enforced in the training tenant).

1. **Information Protection** → **Label policies** → **Publish label**: create policy `TRAIN-LabelPolicy-Pilot`

publishing `TRAIN-Label-Confidential` **only to** `TRAIN-Pilot-Users`.

1. Validate: `OPSTRAIN-LabValidate.ps1 -Lab Labels` confirms the label exists and is published to the

training scope (via `Get-Label` / `Get-LabelPolicy`).

1. **Pass/fail gate:** `TRAIN-Label-Confidential` exists and `TRAIN-LabelPolicy-Pilot` publishes it to

`TRAIN-Pilot-Users` only. A test account sees the label in Office; no real user does.

1. **Teardown:** remove the label policy then the label (portal, or note in the teardown log) to return

to baseline.

Step 6 — Lab 4: Auto-labeling in simulation mode (E5 — verify licensing)

(Run Lab 4 with Lab 1 because it applies the Lab 1 label; sequence it here.)

1. **Information Protection** → **Auto-labeling** → **Create policy**: create `TRAIN-AutoLabel-Sim` that applies `TRAIN-Label-Confidential` based on an **illustrative** condition (e.g. a synthetic sensitive info type), scoped to `TRAIN-Pilot-Users` locations only.

1. **Set the policy to run in *simulation* mode** — do **not** turn it on. (Auto-labeling supports a simulation/"test" mode; `Get-AutoSensitivityLabelPolicy` shows the `Mode`.)

1. Let simulation run, then review the **simulation results** in the portal (matched items, no label actually applied).

1. Validate: `OPSTRAIN-LabValidate.ps1 -Lab AutoLabel` confirms `TRAIN-AutoLabel-Sim` exists and is in a **simulation/test** mode (never enabled).

1. **Pass/fail gate:** the policy exists in **simulation** mode and produced results; **no item was actually labeled. Teardown:** delete `TRAIN-AutoLabel-Sim`.

1. **If auto-labeling is unlicensed:** run as a read-only walkthrough of the portal experience.

Phase D — Data Loss Prevention lab

Step 7 — Lab 2: DLP policy in test / simulation mode (no enforcement)

1. **Purview portal** → **Data Loss Prevention** → **Policies** → **Create policy**: create `TRAIN-DLP-PII-Sim` with an **illustrative** rule (e.g. detect a synthetic PII sensitive info type) across a limited set of locations, scoped to `TRAIN-Pilot-Users` where the workload supports scoping.

1. **Set the policy mode to *Test* (simulation)** — choose **"Test it out first"** (test with or without notifications). Do **not** select *Turn it on / enforce*. In PowerShell this is `Set-DlpCompliancePolicy -Mode TestWithoutNotifications` (modes: `TestWithoutNotifications`, `TestWithNotifications`, `Enable`, `Disable`).

1. Generate a test match with a synthetic-PII document in a training account, then review **DLP alerts / simulation results **— matches are detected,** nothing is blocked**.

1. Validate: `OPSTRAIN-LabValidate.ps1 -Lab DLP` confirms `TRAIN-DLP-PII-Sim` exists and its `Mode` is a **test/simulation** value (asserts it is **not** `Enable`).

1. **Pass/fail gate:** the policy exists in **test** mode, matched the synthetic content, and **enforced nothing. **The trainee can explain how they would move to enforcement and what the blast radius would be.** Teardown:** delete `TRAIN-DLP-PII-Sim` (never leave it enabled).

1. **If advanced DLP is unlicensed:** author a basic policy in test mode, or walk through read-only.

Phase E — Data lifecycle & records lab

Step 8 — Lab 3: Retention policy/label against an illustrative schedule

1. **Purview portal** → **Data Lifecycle Management** → **Retention policies** → **New**: create

`TRAIN-Retention-Pilot` with an **illustrative** period (e.g. "retain 7 years then review" — a *training value, not a real schedule*), scoped to a limited location set / `TRAIN-Pilot-Users`.

1. **Records management** → **Retention labels** → **Create a label**: create `TRAIN-RetLabel-Illustrative`

(do **not** mark it a *record* / *regulatory record* in a shared tenant — discuss what that would do).

1. Validate: `OPSTRAIN-LabValidate.ps1 -Lab Retention` confirms `TRAIN-Retention-Pilot` exists

(`Get-RetentionCompliancePolicy`) and its rule (`Get-RetentionComplianceRule`) carries the illustrative period.

1. **Pass/fail gate:** the retention policy/label exists, scoped and with the **illustrative** period;

the trainee can explain why the **real** schedule is a records/legal determination, not theirs.

1. **Teardown:** remove `TRAIN-Retention-Pilot` and `TRAIN-RetLabel-Illustrative`. *(Note: retention

objects can take time to fully remove — record this as a real operational lesson.)*

Phase F — HR/privacy-sensitive labs (policy-model walkthroughs)

> ***Modules 5 and 6 use NO real employee data.*** *They are taught as* ***policy-model walkthroughs*** *with* ***privacy-by-design***, ***pseudonymization/anonymization where the feature supports it***, *and* ***strict role separation*** *(analyst vs investigator vs admin). All case* ***outcomes*** *are* ***HR/legal determinations*** *— the administrator configures the technical policy, never decides a case.*

Step 9 — Module 5: Insider Risk Management (limited scope, portal)

1. **Purview portal** → **Insider Risk Management**: review **Settings** — turn on/observe **anonymization

(pseudonymized usernames) **as a privacy control, and note the** role separation** (Insider Risk Management Admin vs Analysts vs Investigators).

1. Walk a **policy template** (e.g. data theft / data leaks) at a **limited, illustrative** scope — do

not target real employees; discuss triggering events and the HR/legal review path.

1. **Gate:** the trainee can (a) enable pseudonymization, (b) explain the separated roles, and (c) state

that **case outcomes are HR/legal's determination**, not the administrator's. **Rollback:** leave no policy targeting real users; revert settings changed for the walkthrough.

1. *(Insider Risk configuration is primarily portal-driven; the provided scripts do not write IRM policy —

they only report readable status. Do not invent IRM write cmdlets.)*

Step 10 — Module 6: Communication Compliance (limited scope, portal)

1. **Purview portal** → **Communication Compliance**: review **role separation** (Analyst vs Investigator vs

Admin) and **privacy controls** (pseudonymization of monitored users where supported).

1. Walk a **policy template** against a **training-only** scope — never real communications — and discuss

the review workflow and escalation to HR/legal.

1. **Gate:** the trainee can configure role separation and privacy controls and articulate that

remediation outcomes are HR/legal's, not the administrator's. **Rollback:** no policy over real users; revert walkthrough settings.

Phase G — Audit, eDiscovery, DSPM

Step 11 — Module 7 / Lab 7: Audit search & export (read-only)

1. **Purview portal** → **Audit** (or `Search-UnifiedAuditLog`): run `TRAIN-Audit-Search` for an

illustrative activity over a short window and **export** the results as evidence.

1. Discuss **Audit Standard vs Premium** retention differences (**verify licensing**).

2. Validate: `OPSTRAIN-LabValidate.ps1 -Lab Audit` confirms an audit search returned rows for the window.

3. **Gate:** the trainee runs a search, exports evidence, and explains the Standard/Premium difference.

Rollback: none (read-only).

Step 12 — Module 8 / Lab 8: eDiscovery case + hold on a training custodian (E5 — verify)

1. **Purview portal** → **eDiscovery** → **Cases** → **Create**: create `TRAIN-eDiscovery-Case`.

2. Add a **training custodian** (a `TRAIN-Pilot-Users` test account) and place a **hold**; run a search

in the case. **Never** a real custodian.

1. **Gate:** the case exists, the **training** custodian shows **on hold** (allow up to 24h), and a search

returns. The trainee can explain that a **real** legal hold is a legal determination. **Teardown:** release the hold and close/delete `TRAIN-eDiscovery-Case`.

1. **If eDiscovery Premium is unlicensed:** perform a Standard case, or walk through read-only.

Step 13 — Module 9: DSPM & classification analytics (read-only)

1. **Purview portal** → **DSPM** (Data Security Posture Management) and the **classification / content

explorer / activity explorer **analytics: read the posture and classification picture (**verify licensing/preview**).

1. **Gate:** the trainee can read and interpret the posture/classification analytics and name one action

they would recommend. **Rollback:** none (read-only).

Phase H — Validate competence and operationalize

Step 14 — Validate the full curriculum (pass/fail per lab)

1. Run `OPSTRAIN-LabValidate.ps1 -Lab All` to check every lab's expected object/state in one pass, and

`OPSTRAIN-CompetencyReport.ps1` to roll the results (plus each trainee's created `TRAIN-` objects and completed gates) into a **competency/progress report**.

1. **Gate:** every attempted lab shows **PASS** at its validation, or an explicit, documented reason

(e.g. "unlicensed — walkthrough only"). The report is the **evidence** the team is trained.

Step 15 — Module 10: Turn the labs into operational runbooks

1. Convert each lab into a **runbook** for the recurring work (see Best Practices), e.g.:

• **Weekly DLP alert triage** (review `TRAIN-DLP-PII-Sim`-style alerts → in production, real policy).

• **Monthly label-coverage review** (`OPSTRAIN-LabBaseline.ps1` / `OPSTRAIN-CompetencyReport.ps1`

re-run against production scope, read-only).

• **Retention exception & records requests** → routed to records/legal.

• **Audit search request handling** (`TRAIN-Audit-Search` pattern).

• **Change-control for any new label/DLP/retention policy** — scope-first, simulate-first,

validate-before-enforce.

1. Capture the **RACI** (who configures vs who determines) with the compliance/records/privacy/HR/legal

stakeholders (Requirement 2.7).

1. **Gate:** the runbooks exist, are assigned owners, and route every real determination to the right

governance function.

Step 16 — Teardown & leave the tenant clean

1. Run `OPSTRAIN-LabValidate.ps1 -Lab All` after teardown and compare to the Step 3 baseline: **only**

the baseline objects should remain — **no** `TRAIN-` **object left behind** (except any retention object still in its removal window, which is logged).

1. **Gate:** the tenant matches the baseline (plus documented, in-progress retention removals). Training

is complete and the environment is clean.

Rollback summary

Lab	Teardown
1 Labels	Delete `TRAIN-LabelPolicy-Pilot`, then `TRAIN-Label-Confidential`

Lab	Teardown
4 Auto-labeling	Delete `TRAIN-AutoLabel-Sim` (was only ever in simulation)
2 DLP	Delete `TRAIN-DLP-PII-Sim` (never leave enabled)
3 Retention	Delete `TRAIN-Retention-Pilot` / `TRAIN-RetLabel-Illustrative` (allow removal window)
5 IRM / 6 Comms	Remove any walkthrough policy; revert settings; no real users targeted
7 Audit / 9 DSPM	None (read-only)
8 eDiscovery	Release hold; close/delete `TRAIN-eDiscovery-Case`

Each lab's pass/fail gate and teardown are how the training stays **safe, evidenced, and reversible**.

5. Testing Guide — Microsoft Purview Administrator Training

Run these checks after the labs are configured (Implementation Phases C–G) and again after teardown (Step 16). Each check is a **validation gate** — a lab is only "passed" when its gate is met, and the training is only "complete" when the tenant returns to the Step 3 baseline. Most gates are exercised by `OPSTRAIN-LabValidate.ps1`; `OPSTRAIN-CompetencyReport.ps1` rolls them into the evidence report.

> ***Training, not determination.*** *These tests confirm the *technical* state (object exists, correct mode, correct scope). Whether an *illustrative* taxonomy/period/scope would be *correct in production* is a records/legal/privacy determination — out of scope for the training gate.*
>
> ***Verify in your tenant.*** *Where a feature is unlicensed, its lab is a read-only walkthrough and its gate is "the trainee can navigate and explain it," not "the object exists."*

Test 1 — Readiness: connectivity, scope, and feature visibility

Do: Run `OPSTRAIN-Readiness.ps1`. **Expected (PASS):** `Connect-IPPSSession`/`Connect-MgGraph` succeed; the report names the tenant, shows `TRAIN-Pilot-Users` present, and lists the readable Purview feature areas. Any warned (unreadable/unlicensed) feature is noted so its lab is planned as a walkthrough.

Test 2 — Scope safety: training group contains only test accounts

Do: In **Entra ID** → **Groups** (or `Get-MgGroup` / `Get-MgGroupMember`), inspect `TRAIN-Pilot-Users`. **Expected (PASS):** The group exists and contains **only** training/test accounts — **no real user.** This gate protects every downstream lab.

Test 3 — Lab 1 Labels: label published to the training scope only

Do: `OPSTRAIN-LabValidate.ps1 -Lab Labels`. Cross-check **Information Protection** → **Labels / Label policies**. **Expected (PASS):** `TRAIN-Label-Confidential` exists and `TRAIN-LabelPolicy-Pilot` publishes it to `TRAIN-Pilot-Users` **only**; a test account sees the label in Office, no real user does.

Test 4 — Lab 4 Auto-labeling: exists and is in SIMULATION (never enabled)

Do: `OPSTRAIN-LabValidate.ps1 -Lab AutoLabel`. Cross-check the policy `Mode` (`Get-AutoSensitivityLabelPolicy`). **Expected (PASS):** `TRAIN-AutoLabel-Sim` exists in a **simulation/test** mode and produced results; **no item was actually labeled. FAIL if** the policy is enabled/enforcing. *(Unlicensed → walkthrough; gate is "trainee can explain simulation vs enable.")*

Test 5 — Lab 2 DLP: exists in TEST mode and enforced nothing

Do: `OPSTRAIN-LabValidate.ps1 -Lab DLP`. Cross-check `Get-DlpCompliancePolicy` `Mode`. **Expected (PASS):** `TRAIN-DLP-PII-Sim` exists with `Mode` = a **test/simulation** value (`TestWithoutNotifications`/`TestWithNotifications`), matched the synthetic content, and **blocked nothing. FAIL if** `Mode` = `Enable` — a training DLP policy must never be left enforcing.

Test 6 — Lab 3 Retention: exists, scoped, illustrative period

Do: `OPSTRAIN-LabValidate.ps1 -Lab Retention`. Cross-check `Get-RetentionCompliancePolicy` / `Get-RetentionComplianceRule`. **Expected (PASS):** `TRAIN-Retention-Pilot` (and `TRAIN-RetLabel-Illustrative`) exist, scoped to the training locations, carrying the **illustrative** period; the trainee states the **real** schedule is a records/legal determination.

Test 7 — Modules 5–6: privacy controls and role separation (no real data)

Do: Observe the trainee in the **Insider Risk** and **Communication Compliance** walkthroughs. **Expected (PASS):** The trainee enables **pseudonymization/anonymization**, demonstrates **separated** analyst/investigator/admin roles, targets **no real employees**, and states that **case outcomes are HR/legal's determination. FAIL if** any real user is targeted or role separation is bypassed.

Test 8 — Module 7 Audit: search returns and exports

Do: `OPSTRAIN-LabValidate.ps1 -Lab Audit`. Cross-check the exported `TRAIN-Audit-Search` output. **Expected (PASS):** An audit search over the window returns rows and exports; the trainee explains Audit Standard vs Premium retention. *(No audit data → confirm the window/activity and licensing.)*

Test 9 — Module 8 eDiscovery: training custodian on hold, search returns

Do: In **eDiscovery** → `TRAIN-eDiscovery-Case`, confirm the training custodian's hold status and run a search. **Expected (PASS):** The **training** custodian (a `TRAIN-Pilot-Users` account) shows **on hold** (allow up to 24h) and a search returns; **no real custodian** is held. The trainee states a real hold is a legal determination.

Test 10 — Competency roll-up: every attempted lab PASSes or is documented

Do: `OPSTRAIN-LabValidate.ps1 -Lab All` then `OPSTRAIN-CompetencyReport.ps1`. **Expected (PASS):** Every attempted lab shows **PASS**, or an explicit documented reason (e.g. "unlicensed — walkthrough"). The report is the evidence the team is trained.

Test 11 — Teardown: tenant returns to the Step 3 baseline

Do: After teardown, `OPSTRAIN-LabValidate.ps1 -Lab All` and compare to the `OPSTRAIN-LabBaseline.ps1` baseline CSV. **Expected (PASS): No `TRAIN-` object remains** except any retention object still inside its removal window (logged). The tenant matches the baseline — training left it clean.

Pass/fail summary

#	Gate	PASS condition
1	Readiness	Connected; scope present; feature areas listed
2	Scope safety	`TRAIN-Pilot-Users` = test accounts only
3	Labels	Label published to training scope only
4	Auto-labeling	Exists in **simulation**, nothing labeled
5	DLP	Exists in **test** mode, nothing enforced
6	Retention	Exists, scoped, illustrative period
7	IRM/Comms	Pseudonymization + role separation, no real data
8	Audit	Search returns and exports
9	eDiscovery	Training custodian on hold, search returns
10	Competency	All attempted labs PASS or documented
11	Teardown	Tenant matches baseline

Any FAIL is a stop-and-fix before the trainee is credited with that competency; the enforcement modes (Tests 4 and 5) are the most important — a training policy must **never** be left enabled/enforcing.

6. Troubleshooting Guide — Microsoft Purview Administrator Training

Use `OPSTRAIN-Readiness.ps1` for the current environment/connectivity state, `OPSTRAIN-LabBaseline.ps1` for the known-good "before," `OPSTRAIN-LabValidate.ps1 -Lab …` to re-check a lab's pass/fail gate, and `OPSTRAIN-CompetencyReport.ps1` for the roll-up. Cross-check live state in the **Purview portal**.

> ***Training, not determination.*** *This guide resolves *technical* faults in the training environment (a lab object missing, a wrong mode, connectivity, a scope error). Whether an *illustrative* taxonomy/period is *correct in production* is a records/legal/privacy determination — out of scope.*
>
> ***Verify in your tenant.*** *Purview availability is **licensing-dependent** and paths **evolve** — a "feature not visible" symptom is very often **licensing**, not a fault. Confirm exact paths for your tenant and defer volatile specifics to Microsoft's documentation.*

A. Connectivity & toolchain

Symptom	Cause	Remediation
`OPSTRAIN-Readiness.ps1` says not connected	No S&C PowerShell session	Run `Connect-IPPSSession` (ExchangeOnlineManagement module) with a modern-auth/MFA admin, then re-run
Graph checks skipped/fail	Not connected to Graph, or missing scopes	`Connect-MgGraph` with the scopes the check needs (e.g. group read); consent if prompted
`Get-Label` errors immediately	Session expired or wrong role	Reconnect; confirm the admin holds a Purview role that can read labels (GR §2)
Cmdlet "not recognized"	Module not installed / old version	`Install-Module ExchangeOnlineManagement` / `Install-Module Microsoft.Graph`; update to current

B. Scope group problems (`TRAIN-Pilot-Users`)

Symptom	Cause	Remediation
Readiness reports the group missing	Not yet created	`OPSTRAIN-LabScope.ps1` (preview) then `-Commit`, or create it in **Entra ID** → **Groups**
A real user appears in the group	Wrong member added	Remove immediately — the group must contain **only test accounts**; this gate protects every lab
Label/DLP won't target the group	Group type not supported for that policy, or replication delay	Use a supported group type for the policy; allow directory replication and retry

C. Lab 1 Labels

Symptom	Cause	Remediation
Test account doesn't see the label	Label not published, wrong scope, or client not synced	Confirm `TRAIN-LabelPolicy-Pilot` publishes `TRAIN-Label-Confidential` to `TRAIN-Pilot-Users`; allow policy propagation (can take time); restart the Office client
`OPSTRAIN-LabValidate.ps1 -Lab Labels` FAIL	Label or policy name mismatch	Match names verbatim (`TRAIN-Label-Confidential`, `TRAIN-LabelPolicy-Pilot`); re-check `Get-Label`/`Get-LabelPolicy`
A real user saw the label	Policy scoped too broadly	Narrow the policy to `TRAIN-Pilot-Users` **only**; unpublish and republish scoped

D. Lab 2 DLP — the critical "never enforce" checks

Symptom	Cause	Remediation
Validate FAIL: `Mode = Enable`	Policy was turned on instead of test	**Set it back to test** (`Set-DlpCompliancePolicy -Mode TestWithoutNotifications`) — a training DLP policy must never enforce; or delete it
No match on the synthetic document	Rule condition / sensitive info type mismatch	Confirm the rule's sensitive info type matches the synthetic content; allow policy sync before testing
Content was actually blocked	Policy was enforcing	This is a FAIL — revert to test mode or delete; investigate why it was enabled
Advanced DLP options missing	Unlicensed	Author a basic policy in test mode, or run read-only (**verify licensing**)

E. Lab 4 Auto-labeling — simulation only

Symptom	Cause	Remediation
Validate FAIL: not in simulation	Policy enabled	Return to **simulation** (never enable a training auto-label policy) or delete `TRAIN-AutoLabel-Sim`

Symptom	Cause	Remediation
No simulation results	Not enough matching content, or run not finished	Add synthetic matching content; allow the simulation to complete (can take time)
Auto-labeling not available	Unlicensed (E5)	Walkthrough only; set the gate to "explain simulation vs enable" (**verify licensing**)

F. Lab 3 Retention

Symptom	Cause	Remediation
Policy won't delete at teardown	Retention objects have a removal window	Expected — log it; retention policies/labels can take time to fully remove (a real operational lesson)
Can't mark the label a record	Records management not licensed, or intentionally avoided	Don't mark records in a shared training tenant; discuss what it would do (**verify licensing**)
Validate FAIL	Name/period mismatch	Match `TRAIN-Retention-Pilot`/`TRAIN-RetLabel-Illustrative`; confirm the illustrative period via `Get-RetentionComplianceRule`

G. Modules 5–6 Insider Risk / Communication Compliance

Symptom	Cause	Remediation
Feature not visible	Unlicensed (E5)	Walkthrough only; gate = "explain the policy model, privacy controls, role separation" (**verify licensing**)
Can't separate roles	One person holds all roles	Assign analyst / investigator / admin to **different** people — role separation is a privacy control, not optional
Anonymization off	Setting not enabled	Turn on **pseudonymization/anonymization** before any walkthrough; never use real employee data
Pressure to "resolve a case" in training	Confusing configure with determine	Stop — **case outcomes are HR/legal's determination**; training only covers the technical policy config

H. Module 7 Audit

Symptom	Cause	Remediation
Search returns nothing	Window too narrow, activity not logged yet, or auditing off	Widen the window; confirm unified audit logging is on; generate then search for a known activity
Older events missing	Audit **Standard** retention limit	Expected — discuss **Audit Premium** longer retention (**verify licensing**)

I. Module 8 eDiscovery

Symptom	Cause	Remediation
Custodian not on hold	Holds can take up to 24h; or licensing	Wait up to 24h; confirm the **training** custodian's license supports being held; re-check the case
A real custodian was added	Wrong account	Remove immediately — hold **only** a `TRAIN-Pilot-Users` test account; never a real custodian
Premium features missing	Unlicensed	Run a Standard case or walkthrough (**verify licensing**)

J. Validation / competency roll-up

Symptom	Cause	Remediation
`-Lab All` shows unexpected FAILs	Name mismatch or feature unlicensed	Match object names verbatim; for unlicensed features record "walkthrough only" as the documented reason
Teardown check still shows `TRAIN-` objects	Not fully removed, or retention in its window	Remove remaining objects; log any retention object still inside its removal window
Report empty	Validation not run first	Run `OPSTRAIN-LabValidate.ps1 -Lab All` before `OPSTRAIN-CompetencyReport.ps1`

When to escalate to the governance functions (not a "fix")

- A trainee asks what the **real** retention period, records classification, legal hold scope, or

insider-risk/communication-compliance **case outcome** should be → that is **not** a training fault to fix. Route it to **records / legal / privacy / HR** — the training deliberately uses illustrative values and defers the determination.

7. Best Practices — Microsoft Purview Administrator Training

Field-proven recommendations for building durable Purview administrator competence — and for the operational habits the training must instill.

- **Train the safe-rollout pattern, not just the feature.** The single most valuable thing an

administrator can internalize is **scope-first, simulate-first, validate-before-enforce**. Every lab is built that way on purpose — a training group, simulation/test/disabled mode, a pass/fail gate — so the habit transfers directly to production, where DLP, labels, retention, and auto-labeling are all high-impact.

- **Never let a training policy enforce.** DLP stays in **test** mode; auto-labeling stays in

simulation; retention uses **illustrative** periods; eDiscovery holds only a **training** custodian. The validation gates explicitly FAIL an enabled/enforcing training policy — treat that as a hard stop, not a warning.

- **Prefer a sandbox/training tenant.** A dedicated tenant removes all doubt. When you must train in

production, confine every lab to `TRAIN-Pilot-Users` and simulation/test, and **tear down to the baseline** afterward — the `OPSTRAIN-LabBaseline.ps1` snapshot is what "clean" means.

- **Use the `TRAIN-` naming convention verbatim.** Consistent, prefixed names make every training

artifact unmistakable, let `OPSTRAIN-LabValidate.ps1` find and check it, and make teardown reliable — the same object-naming discipline production Purview work needs.

- **Teach least privilege from Module 0.** Have each trainee operate from the **least-privilege role

group** for their module, with Global Administrator reserved. Learning to do the job without over-permission is itself a core competency and a security control.

- **Keep Insider Risk and Communication Compliance privacy-first.** These are HR/privacy-sensitive.

Always enable **pseudonymization/anonymization**, enforce **strict role separation** (analyst vs investigator vs admin — different people), use **no real employee data** in training, and make clear that **case outcomes are HR/legal's determination**, never the administrator's.

- **Draw the configure-vs-determine line explicitly.** The administrator *configures the technical

control*; the *records schedule, file plan, legal hold, and case outcome* are decided by compliance/records/privacy/HR/legal. Bake this into every relevant lab and into the Module 10 RACI so a trained admin never mistakes a mechanic for a mandate.

- **Validate competence with evidence, not attendance.** "Trained" means each lab's pass/fail gate was

met and captured in the `OPSTRAIN-CompetencyReport.ps1` output — an evidenced capability, re-runnable when you onboard the next administrator.

- **Turn labs into runbooks.** The value compounds when Module 10 converts the labs into the recurring

operational work — weekly DLP alert triage, monthly label-coverage review, retention/records request routing, audit-search handling, and change-control for any new policy. Assign owners; the runbook is the deliverable, not the memory of the lab.

- **Verify licensing per module, and degrade gracefully.** Advanced DLP, auto-labeling, insider risk,

communication compliance, Audit Premium, eDiscovery Premium, and DSPM need **E5 / E5 Compliance** (some preview). Where a feature is unlicensed, run a **read-only walkthrough** and set the gate to "can navigate and explain" — never assume a capability is present.

- **Verify evolving portal surfaces in your tenant.** Purview paths and cmdlet behavior change. Confirm exact toggles for your tenant, flag anything volatile **"verify in your tenant,"** and defer to Microsoft's current documentation — never invent a path or cmdlet to make a lab "work."
- **Re-run the curriculum for every new hire.** The whole package is reusable: run readiness, stand up the scope, run the labs, validate, tear down. Onboarding a new Purview administrator becomes a known, evidenced, low-risk process instead of another round of trial-and-error on production.

8. Deliverables Summary — Microsoft Purview Administrator Training

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box
1. Solution Overview	The training model — a lab-and-runbook curriculum that teaches every Purview solution area **scope-first, simulate-first, validate-before-enforce**
2. Requirements Guide	Every prerequisite with Verify / Implement-if-missing (training environment, scope group, least-privilege roles, toolchain, licensing, illustrative data, stakeholders)
Global Requirements (Purview)	Category-wide prerequisites, published once
3. Implementation Guide	Ordered steps: stand up the program (readiness, scope, baseline), then run each lab in a training scope and simulation/test mode with a pass/fail gate and teardown, then operationalize as runbooks
5. Testing Guide	Per-lab validation gates with explicit PASS conditions, plus the teardown-to-baseline check
6. Troubleshooting Guide	Field issues across labs and their remediation
7. Best Practices	Field-proven recommendations for building durable, evidenced Purview competence

PowerShell scripts (Security & Compliance PowerShell / Microsoft Graph — safe by default)

Script	What it does					
`OPSTRAIN-Readiness.ps1`	**Scope/health helper (read-only):** confirms `Connect-IPPSSession`/`Connect-MgGraph`, reports the tenant, role context, whether `TRAIN-Pilot-Users` exists, and which Purview feature areas are readable (warns where a feature is unlicensed → walkthrough)					
`OPSTRAIN-LabScope.ps1`	**Configuration helper (default `-WhatIf`; `-Commit` to apply):** creates/verifies the `TRAIN-Pilot-Users` training scope group of test accounts; the only script that writes, and only the scope group					
`OPSTRAIN-LabBaseline.ps1`	**Discovery (read-only):** snapshots the current labels, label policies, DLP policies/rules, retention policies, and auto-labeling policies to a baseline CSV — the known-good "before" state for teardown					
`OPSTRAIN-LabValidate.ps1`	**Validation (read-only):** checks each lab's expected object and **mode/scope** — labels published to the training scope, DLP in **test** mode (not `Enable`), auto-labeling in **simulation**, retention scoped with the illustrative period, audit search returns — pass/fail per lab (`-Lab Labels	DLP	Retention	AutoLabel	Audit	All `)
`OPSTRAIN-CompetencyReport.ps1`	**Reporting (read-only):** rolls the validation results and each trainee's `TRAIN-` objects/gates into a competency/progress report CSV — the evidence the team is trained					

All scripts are **read-only by default** (only `OPSTRAIN-LabScope.ps1` writes, and only with `-Commit`, and only the training scope group). No credentials or secrets are stored. Cmdlets are standard Security & Compliance PowerShell / Microsoft Graph — none are invented; Insider Risk / Communication Compliance policy writes are **not** scripted (portal-driven, HR/privacy-sensitive).

What the buyer receives

- This curriculum (Solution Overview, Requirements, Implementation, Testing, Troubleshooting, Best Practices) as a **Word** document and **PDF**.
- All five **PowerShell scripts** in a single ZIP.

- A repeatable, evidence-producing training process following the model:

Assess → **Design** → **Configure** → **Pilot (simulate/test)** → **Validate** → **Expand.**

Scope note

This package delivers **administrator skills training** — the *technical method* for operating every Microsoft Purview solution area, practiced safely on **illustrative** data in a scoped, simulated sandbox. It is **not** a records, retention, privacy, or legal determination: real retention schedules, records file plans, legal holds, and insider-risk / communication-compliance **case outcomes** are **deferred** to the organization's compliance, records, privacy, HR, and legal functions. Purview is **licensing-dependent** and the portal **evolves** — availability and paths are **cited or flagged "verify your licensing/tenant,"** never invented.

Outcome

A Purview administration team that has **built and proven** competence across labels, DLP, data lifecycle & records, insider risk, communication compliance, audit, eDiscovery, and DSPM — each practiced scope-first and simulate-first, validated at a pass/fail gate, and converted into operational runbooks — with the whole curriculum reusable for every new administrator you onboard, and the training tenant left clean.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

OPSTRAIN-CompetencyReport.ps1

Purpose: Reporting for the training curriculum. Reads the validation output produced by OPSTRAIN-LabValidate.ps1 (default .\OPSTRAIN-Validation.csv) and inventories the TRAIN- objects a trainee created (label, label policy, DLP-sim policy, retention policy, auto-labeling-sim policy) from Security & Compliance PowerShell, then rolls both into a single competency report: per-lab PASS/FAIL, the object that evidences it, and an overall trained/not-yet signal. Read-only — makes no changes. This is the EVIDENCE the team is trained; it does not credit competency itself (that is the trainer/owner's determination). HOW TO RUN PowerShell 5.1+ or 7+ with Security & Compliance PowerShell (Connect-IPPSSession): Connect-IPPSSession .\OPSTRAIN-LabValidate.ps1 -Lab All # run FIRST - produces OPSTRAIN-Validation.csv .\OPSTRAIN-CompetencyReport.ps1 .\OPSTRAIN-CompetencyReport.ps1 -Trainee "jdoe" -ValidationCsv .\OPSTRAIN-Validation.csv -OutPath .\OPSTRAIN-Competency.csv EXPECTED RESULTS A per-lab competency table (lab, expected TRAIN- object, exists?, validation status) and an overall summary written to .\OPSTRAIN-Competency.csv. If the validation CSV is missing, the report says so and tells you to run OPSTRAIN-LabValidate.ps1 first (an empty report means validation was not run). Nothing is modified. .NOTES White-label Microsoft Purview training tooling. Read-only. No credentials stored. Cmdlets are standard Security & Compliance PowerShell; none are invented. Object names default to the TRAIN- convention used in the Implementation Guide — override with the parameters if you named yours differently. Crediting a competency is a trainer/owner determination; this script only produces the evidence. > [CmdletBinding()] param([string]$Trainee = "", [string]$ValidationCsv = ".\OPSTRAIN-Validation.csv", [string]$LabelName = "TRAIN-Label-Confidential", [string]$LabelPolicyName = "TRAIN-LabelPolicy-Pilot", [string]$DlpPolicyName = "TRAIN-DLP-PII-Sim", [string]$RetentionName = "TRAIN-Retention-Pilot", [string]$AutoLabelName = "TRAIN-AutoLabel-Sim", [string]$OutPath = ".\OPSTRAIN-Competency.csv")

How to run: PowerShell 5.1+ or 7+ with Security & Compliance PowerShell (Connect-IPPSSession): Connect-IPPSSession .\OPSTRAIN-LabValidate.ps1 -Lab All # run FIRST - produces OPSTRAIN-Validation.csv .\OPSTRAIN-CompetencyReport.ps1 .\OPSTRAIN-CompetencyReport.ps1 -Trainee "jdoe" -ValidationCsv .\OPSTRAIN-Validation.csv -OutPath .\OPSTRAIN-Competency.csv EXPECTED RESULTS A per-lab competency table (lab, expected TRAIN- object, exists?, validation status) and an overall summary written to .\OPSTRAIN-Competency.csv. If the validation CSV is missing, the report says so and tells you to run OPSTRAIN-LabValidate.ps1 first (an empty report means validation was not run). Nothing is modified. .NOTES White-label Microsoft Purview training tooling. Read-only. No credentials stored. Cmdlets are standard Security & Compliance PowerShell; none are invented. Object names default to the TRAIN- convention used in the Implementation Guide — override with the parameters if you named yours differently. Crediting a competency is a trainer/owner determination; this script only produces the evidence. > [CmdletBinding()] param([string]$Trainee = "", [string]$ValidationCsv = ".\OPSTRAIN-Validation.csv", [string]$LabelName = "TRAIN-Label-Confidential", [string]$LabelPolicyName = "TRAIN-LabelPolicy-Pilot", [string]$DlpPolicyName = "TRAIN-DLP-PII-Sim", [string]$RetentionName = "TRAIN-Retention-Pilot", [string]$AutoLabelName = "TRAIN-AutoLabel-Sim", [string]$OutPath = ".\OPSTRAIN-Competency.csv")

OPSTRAIN-LabBaseline.ps1

Purpose: Discovery for the training curriculum. Enumerates the Purview objects the labs will add to — sensitivity labels, label policies, DLP policies and rules, retention policies, and auto-labeling policies — and writes them to a baseline CSV. This is the known-good state each lab teardown returns to, and the reference OPSTRAIN-LabValidate.ps1 diffs against to confirm only TRAIN- objects were added. Run it BEFORE Lab 1 and again after teardown. Changes nothing. HOW TO RUN PowerShell 5.1+ or 7+ with Security & Compliance PowerShell (Connect-IPPSSession): Connect-IPPSSession .\OPSTRAIN-LabBaseline.ps1 .\OPSTRAIN-LabBaseline.ps1 -OutPath ".\OPSTRAIN-Baseline-before.csv" EXPECTED RESULTS The full inventory to .\OPSTRAIN-Baseline.csv (ArtifactType,Name,Detail). Compare a post-teardown run against the pre-training run: the only difference should be TRAIN- objects added then removed. .NOTES White-label Microsoft Purview training tooling. Read-only. No credentials stored. Cmdlets are standard Security & Compliance PowerShell; none are invented. Feature availability is licensing-dependent — the script WARNS where an area is not readable rather than assuming it exists. > [CmdletBinding()] param([string]$OutPath = ".\OPSTRAIN-Baseline.csv")

How to run: PowerShell 5.1+ or 7+ with Security & Compliance PowerShell (Connect-IPPSSession): Connect-IPPSSession .\OPSTRAIN-LabBaseline.ps1 .\OPSTRAIN-LabBaseline.ps1 -OutPath ".\OPSTRAIN-Baseline-before.csv" EXPECTED RESULTS The full inventory to .\OPSTRAIN-Baseline.csv (ArtifactType,Name,Detail). Compare a post-teardown run against the pre-training run: the only difference should be TRAIN- objects added then removed. .NOTES White-label Microsoft Purview training tooling. Read-only. No credentials stored. Cmdlets are standard Security & Compliance PowerShell; none are invented. Feature availability is licensing-dependent — the script WARNS where an area is not readable rather than assuming it exists. > [CmdletBinding()] param([string]$OutPath = ".\OPSTRAIN-Baseline.csv")

OPSTRAIN-LabScope.ps1

Purpose: The one configuration helper in the curriculum. Ensures the training scope group (default TRAIN-Pilot-Users) exists in Entra ID via Microsoft Graph, so every lab can target it and never a real user. By DEFAULT it only PREVIEWS (-WhatIf) — it reports whether the group exists and what it would create. With -Commit it creates the group if missing (as a security group). It never adds members automatically and never touches any Purview policy — it only provisions the empty scope container; you add TEST accounts deliberately. HOW TO RUN PowerShell 5.1+ or 7+ with Microsoft Graph (group write scope needed only for -Commit): Connect-MgGraph -Scopes "Group.ReadWrite.All" .\OPSTRAIN-LabScope.ps1 # preview only (default -WhatIf) .\OPSTRAIN-LabScope.ps1 -Commit # create the group if missing .\OPSTRAIN-LabScope.ps1 -ScopeGroup "TRAIN-Pilot-Users" -Commit EXPECTED RESULTS Preview: reports "would create" or "already exists". -Commit: the group exists afterward and its Id is printed. PASS = TRAIN-Pilot-Users exists; then add ONLY test accounts to it before Lab 1. .NOTES White-label Microsoft Purview training tooling. Writes ONLY with -Commit, and ONLY the scope group (no members, no policies). No credentials stored. Cmdlets are standard Microsoft Graph PowerShell; none are invented. > [CmdletBinding()] param([string]$ScopeGroup = "TRAIN-Pilot-Users", [switch]$Commit)

How to run: PowerShell 5.1+ or 7+ with Microsoft Graph (group write scope needed only for -Commit): Connect-MgGraph -Scopes "Group.ReadWrite.All" .\OPSTRAIN-LabScope.ps1 # preview only (default -WhatIf) .\OPSTRAIN-LabScope.ps1 -Commit # create the group if missing .\OPSTRAIN-LabScope.ps1 -ScopeGroup "TRAIN-Pilot-Users" -Commit EXPECTED RESULTS Preview: reports "would create" or "already exists". -Commit: the group exists afterward and its Id is printed. PASS = TRAIN-Pilot-Users exists; then add ONLY test accounts to it before Lab 1. .NOTES White-label Microsoft Purview training tooling. Writes ONLY with -Commit, and ONLY the scope group (no members, no policies). No credentials stored. Cmdlets are standard Microsoft Graph PowerShell; none are invented. > [CmdletBinding()] param([string]$ScopeGroup = "TRAIN-Pilot-Users", [switch]$Commit)

OPSTRAIN-LabValidate.ps1

Purpose: Validation for the training curriculum. For each lab, confirms the expected TRAIN- object exists in the CORRECT mode/scope — Lab 1 label published to the training scope; Lab 2 DLP in a TEST/simulation mode (asserts NOT Enable — a training DLP policy must never enforce); Lab 4 auto-labeling in a simulation/test mode (asserts NOT enabled); Lab 3 retention present; Lab 7 audit search returns rows. Emits PASS/FAIL per lab. Choose a lab or run them all. Changes nothing. HOW TO RUN PowerShell 5.1+ or 7+ with Security & Compliance PowerShell (Connect-IPPSSession): Connect-IPPSSession .\OPSTRAIN-LabValidate.ps1 -Lab All .\OPSTRAIN-LabValidate.ps1 -Lab DLP .\OPSTRAIN-LabValidate.ps1 -Lab Labels -ScopeGroup "TRAIN-Pilot-Users" EXPECTED RESULTS A pass/fail line per checked lab plus a summary, written to .\OPSTRAIN-Validation.csv. The critical gates are DLP (must be TEST, never Enable) and AutoLabel (must be SIMULATION, never enabled). A FAIL is a stop-and-fix before crediting that competency. .NOTES White-label Microsoft Purview training tooling. Read-only. No credentials stored. Cmdlets are standard Security & Compliance PowerShell; none are invented. Object names default to the TRAIN- convention used in the Implementation Guide — override with the parameters if you named yours differently. > [CmdletBinding()] param([ValidateSet("Labels","DLP","Retention","AutoLabel","Audit","All")] [string]$Lab = "All", [string]$ScopeGroup = "TRAIN-Pilot-Users", [string]$LabelName = "TRAIN-Label-Confidential", [string]$LabelPolicyName = "TRAIN-LabelPolicy-Pilot", [string]$DlpPolicyName = "TRAIN-DLP-PII-Sim", [string]$RetentionName = "TRAIN-Retention-Pilot", [string]$AutoLabelName = "TRAIN-AutoLabel-Sim", [string]$OutPath = ".\OPSTRAIN-Validation.csv")

How to run: PowerShell 5.1+ or 7+ with Security & Compliance PowerShell (Connect-IPPSSession): Connect-IPPSSession .\OPSTRAIN-LabValidate.ps1 -Lab All .\OPSTRAIN-LabValidate.ps1 -Lab DLP .\OPSTRAIN-LabValidate.ps1 -Lab Labels -ScopeGroup "TRAIN-Pilot-Users" EXPECTED RESULTS A pass/fail line per checked lab plus a summary, written to .\OPSTRAIN-Validation.csv. The critical gates are DLP (must be TEST, never Enable) and AutoLabel (must be SIMULATION, never enabled). A FAIL is a stop-and-fix before crediting that competency. .NOTES White-label Microsoft Purview training tooling. Read-only. No credentials stored. Cmdlets are standard Security & Compliance PowerShell; none are invented. Object names default to the TRAIN- convention used in the Implementation Guide — override with the parameters if you named yours differently. > [CmdletBinding()] param([ValidateSet("Labels","DLP","Retention","AutoLabel","Audit","All")] [string]$Lab = "All", [string]$ScopeGroup = "TRAIN-Pilot-Users", [string]$LabelName = "TRAIN-Label-Confidential", [string]$LabelPolicyName = "TRAIN-LabelPolicy-Pilot", [string]$DlpPolicyName = "TRAIN-DLP-PII-Sim", [string]$RetentionName = "TRAIN-Retention-Pilot", [string]$AutoLabelName = "TRAIN-AutoLabel-Sim", [string]$OutPath = ".\OPSTRAIN-Validation.csv")

OPSTRAIN-Readiness.ps1

Purpose: The scope/health helper for the Purview Administrator Training curriculum. Confirms the training workstation is connected to Security & Compliance PowerShell (and, if available, Microsoft Graph), reports the connected tenant and the readable role context, checks whether the training scope group (default TRAIN-Pilot-Users) exists, and probes which Purview feature areas are READABLE (labels, DLP, retention, auto-labeling, audit) — warning where a feature is not readable (very often LICENSING, which means that lab runs as a read-only walkthrough). Changes nothing. HOW TO RUN PowerShell 5.1+ or 7+ with Security & Compliance PowerShell (Connect-IPPSSession); Microsoft Graph optional for the scope-group check: Connect-IPPSSession Connect-MgGraph -Scopes "Group.Read.All" # optional, enables the scope-group check .\OPSTRAIN-Readiness.ps1 .\OPSTRAIN-Readiness.ps1 -ScopeGroup

"TRAIN-Pilot-Users" EXPECTED RESULTS A readiness summary to the console and to .\OPSTRAIN-Readiness.csv. PASS = connected, scope status reported, and the feature areas your curriculum covers are listed READABLE. Any WARN feature is planned as a walkthrough, not a failure. .NOTES White-label Microsoft Purview training tooling. Read-only. No credentials stored. Cmdlets are standard Security & Compliance PowerShell / Microsoft Graph; none are invented. Feature availability is licensing-dependent — the script WARNS where a feature is not readable rather than assuming it exists. > [CmdletBinding()] param([string]$ScopeGroup = "TRAIN-Pilot-Users", [string]$OutPath = ".\OPSTRAIN-Readiness.csv")

How to run: PowerShell 5.1+ or 7+ with Security & Compliance PowerShell (Connect-IPPSSession); Microsoft Graph optional for the scope-group check: Connect-IPPSSession Connect-MgGraph -Scopes "Group.Read.All" # optional, enables the scope-group check .\OPSTRAIN-Readiness.ps1 .\OPSTRAIN-Readiness.ps1 -ScopeGroup "TRAIN-Pilot-Users" EXPECTED RESULTS A readiness summary to the console and to .\OPSTRAIN-Readiness.csv. PASS = connected, scope status reported, and the feature areas your curriculum covers are listed READABLE. Any WARN feature is planned as a walkthrough, not a failure. .NOTES White-label Microsoft Purview training tooling. Read-only. No credentials stored. Cmdlets are standard Security & Compliance PowerShell / Microsoft Graph; none are invented. Feature availability is licensing-dependent — the script WARNS where a feature is not readable rather than assuming it exists. > [CmdletBinding()] param([string]$ScopeGroup = "TRAIN-Pilot-Users", [string]$OutPath = ".\OPSTRAIN-Readiness.csv")